

Smart Construction Connect

Infrastructure, storage architecture and containerisation handbook — everything needed to build images and deploy this application on Kubernetes / EKS.

Repository: <https://github.com/MuhammadAzan169/Smart-Construction-Connect> · **Branch:** main · **Document date:** 17 August 2026

Prepared for: DevOps containerisation / EKS practice deployment. Every answer below was verified against the source code, not the README; file and line references are given throughout.

READ THIS FIRST

There are no Dockerfiles in this repository. No `Dockerfile`, no `docker-compose.yml`, no `.dockerignore` exists in any directory. The application currently deploys to managed platforms (Vercel + Render) that build from source, so container definitions were never needed. Section 6 gives complete, ready-to-use reference Dockerfiles that match how the platforms actually build and run the app.

Contents

- 1. What the application is
- 2. Current production deployment
- 3. Storage architecture — the four directories answered (A–E)
- 4. What this means for EKS: EBS vs S3 vs Supabase vs ephemeral
- 5. Runtime characteristics that affect Kubernetes
- 6. Containerisation: reference Dockerfiles and build commands
- 7. Environment variables reference
- 8. Networking, routing and ingress
- 9. Health checks, probes and resources
- 10. Known gotchas

1. What the application is

A construction marketplace and AI assistant platform. Clients, construction companies, material suppliers and administrators each get a role-scoped dashboard. Features: JWT authentication, company/supplier discovery and verification, project requests, real-time chat, document upload with text extraction, an English/Urdu bilingual UI, and a retrieval-augmented AI assistant.

It is a **two-part application in one repository**:

Part	Technology	Description
Frontend/	React 18, TypeScript, Vite 7, Tailwind CSS, shadcn/ui	Static single-page application. Build output is a folder of static files (<code>dist/</code>) — no Node.js runtime is required in production, only a static web server.
Backend/	Python 3.11.9, FastAPI, Uvicorn	Stateless HTTP API under <code>/api/...</code> , plus one WebSocket endpoint and one Server-Sent Events endpoint. Single process, ASGI.

There is no separate worker, queue, cron job or cache service. The full runtime is: one Python process, one static file server, plus the managed services listed next.

2. Current production deployment

Component	Platform	Details
Frontend SPA	Vercel	Root directory <code>Frontend/</code> , build <code>vite build</code> , output <code>dist/</code> . Live at <code>https://smart-construction-connect-frontend.vercel.app</code>
Backend API	Render (free web service)	Root directory <code>Backend/</code> , start <code>uvicorn app:app --host 0.0.0.0 --port \$PORT</code> . Live at <code>https://smart-construction-connect-backend.onrender.com</code>
Database	Supabase Postgres	Connected via the transaction pooler (pgBouncer), port 6543. One table: <code>documents(key text pk, data jsonb, updated_at timestampz)</code>
Binary file storage	Supabase Storage	One public bucket, default name <code>media</code> . Accessed over its REST API with the service-role key.
Embeddings (search)	Cohere	OpenAI-compatible endpoint <code>https://api.cohere.ai/compatibility/v1</code> , model <code>embed-multilingual-light-v3.0</code> . Optional — see gotchas.
LLM chat	OpenRouter	Free model chain with automatic key and model rotation on rate limits.

All six are reachable over plain HTTPS from anywhere, so **the same architecture works unchanged inside Kubernetes** — the pods simply need outbound internet (NAT gateway) and the secrets injected as environment variables.

3. Storage architecture — the four directories answered

The backend behaves differently depending on which environment variables are set. This is the single most important thing to understand before deploying it:

Configuration	JSON documents go to	Binary files go to
<code>DATABASE_URL</code> empty and <code>SUPABASE_*</code> empty <i>(local development mode)</i>	Local files under <code>Backend/Database/</code>	Local disk under <code>Backend/company_data/</code> and <code>Backend/uploads/</code>
<code>DATABASE_URL</code> + <code>SUPABASE_URL</code> + <code>SUPABASE_SERVICE_ROLE_KEY</code> set <i>(production mode — what you should deploy)</i>	Supabase Postgres, table <code>documents</code>	Supabase Storage bucket

The switch is not a code branch you have to enable — it is automatic and checked at every read and write.

A. Is `Database/*.json` still the actual production database?

No — not in production. It is the local-development store and the migration source. In production every read and write goes to Supabase Postgres.

Every JSON read and write in the entire codebase funnels through two functions, `read_json()` and `write_json()`. Both check whether Postgres is enabled before touching the disk:

```
def read_json(filepath: Path) -> Any:
    from backend.utils import db
    if db.is_enabled():
        # DATABASE_URL is set and connected
        val = db.kv_get(_doc_key(filepath)) # -> SELECT data FROM documents WHERE key = %s
        return val if val is not None else []
    ... # otherwise: read the local file

def write_json(filepath: Path, data: Any):
    from backend.utils import db
    if db.is_enabled():
        db.kv_set(_doc_key(filepath), data) # -> INSERT ... ON CONFLICT DO UPDATE
        return
    ... # otherwise: atomic temp-file + os.replace
```

Source: `Backend/backend/utils/data_handler.py` lines 89–132; `Backend/backend/utils/db.py` lines 117–157.

The Postgres row key is the document's old file path relative to the backend root — for example `Database/clients/users.json` becomes the primary key of one row whose `data` column holds the whole JSON array. So the directory tree still exists conceptually, but as keys in one table.

Consequence for you: the `Database/` directory that ships in the repository contains seeded demo data (fictional companies, users, chat history). Bake it into the image if you like — it is a few megabytes and is only read when Postgres is disabled — but in production it is dead weight and is never written to. **It needs no persistent volume.**

B. Does `company_data/` still serve files directly from the container?

Partly — and this is the one genuine subtlety in the whole system.

Two different things share this path:

- **Pre-existing demo assets committed to the repository** (company logos, gallery images, sample documents) are still served directly by the container. `app.py` mounts the directory as static files:

```
COMPANY_DATA_DIR = ROOT / "company_data"
app.mount("/company_data", StaticFiles(directory=str(COMPANY_DATA_DIR)), name="static-company-data")

UPLOADS_DIR = ROOT / "uploads"
app.mount("/uploads", StaticFiles(directory=str(UPLOADS_DIR)), name="static-uploads")
```

Source: `Backend/app.py` lines 186–196. These files are read-only, they come from the git checkout, and they are identical in every replica. That makes them **image content, not state**.

- **Newly uploaded files** (profile images, verification documents, gallery uploads) do *not* land there in production. They go to Supabase Storage and the API returns an absolute Supabase CDN URL that the browser fetches directly, bypassing the backend entirely.

```
def save_bytes(rel_path: str, data: bytes, content_type: str = "...") -> str:
    if is_remote():
        # SUPABASE_URL and SUPABASE_SERVICE_ROLE_KEY both set
        endpoint = f"{url}/storage/v1/object/{bucket}/{rel_path}"
        resp = httpx.post(endpoint, headers={"Authorization": f"Bearer {key}", ...}, content=data)
        return f"{url}/storage/v1/object/public/{bucket}/{rel_path}" # absolute CDN URL
    # — Local disk fallback (development only) —
    dest = _REPO_ROOT / rel_path
    dest.write_bytes(data)
    return f"/{rel_path}" # relative URL served by the StaticFiles mount
```

Source: Backend/backend/utils/storage.py lines 41–78.

Consequence for you: keep the `/company_data` route working (old records still reference relative URLs), but it needs **no shared or persistent volume** — the content is baked into the image at build time. Nothing writes to it once Supabase is configured.

C. Does `uploads/` receive user uploads during runtime?

Only in local development. In production, message attachments and voice notes go to Supabase Storage through the same `save_bytes()` function shown above.

The directory is created at startup and mounted as static files purely so that development works and so that any legacy relative URLs still resolve. Note that `uploads/*` is git-ignored — the directory ships empty apart from a marker file.

There is one related piece of state worth knowing about: so the admin file monitor still works when the binaries live in Supabase rather than on a disk the backend can walk, an index of uploads is kept as a JSON document (`Database/uploads_index.json`) — which means it too lives in Postgres in production.

Source: Backend/backend/utils/data_handler.py lines 266–292.

Consequence for you: ephemeral filesystem is correct. No volume needed.

D. Is `index_data/` just a rebuildable cache?

Yes. Entirely rebuildable, derived data — safe to lose at any time.

It holds exactly two artefacts: `faiss.index` (vector index) and `meta.pkl` (BM25 state, entity records and content hashes). Both are derived from the company and supplier data. The directory is git-ignored and the index is built during application startup:

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    count = initialize_embeddings() # builds the hybrid semantic index
    response_cache.invalidate()
    stop_watcher = start_background_watcher() # polls the source data every 10s for changes
    yield
    stop_watcher()
```

Source: Backend/app.py lines 116–136; Backend/backend/utils/semantic_embeddings.py lines 47–49;
Backend/backend/utils/index_sync.py.

Consequence for you: ephemeral filesystem, and never share it between pods. Each replica should build and own its own copy. Sharing a FAISS index and a pickle file across pods over a network volume would invite corruption for no benefit. The only cost of not persisting it is startup time — see section 9.

E. Which of these have already moved to Supabase?

Directory	Production destination	Written at runtime?	Volume required
Database/	Supabase Postgres (table documents)	Yes — but to Postgres, not to disk	None
company_data/	New uploads → Supabase Storage . Committed demo assets → served from the image.	No (read-only in production)	None
uploads/	Supabase Storage	No (production writes go to Supabase)	None
index_data/	Stays local — rebuilt in each pod at startup	Yes, but disposable cache only	None (ephemeral, per-pod)

4. What this means for EKS: EBS vs S3 vs Supabase vs ephemeral

SHORT ANSWER

Ephemeral filesystem. No EBS, no EFS, no S3, no PersistentVolumeClaim. Provided `DATABASE_URL` , `SUPABASE_URL` and `SUPABASE_SERVICE_ROLE_KEY` are set, all durable state already lives in Supabase and the backend pod is stateless. It can be a plain `Deployment` — a `StatefulSet` would buy nothing.

Option	Needed?	Reasoning
EBS	No	Nothing durable is written to the local disk. EBS volumes are also single-attach (ReadWriteOnce), which would cap the deployment at one replica per volume for no gain.
EFS	No	Only relevant if pods had to share writable state. They do not — Postgres and Supabase Storage already do that job.
S3	No, but a valid exercise	Not required, because Supabase Storage already handles binaries. If you want S3 practice, the swap is small and well isolated: <code>storage.py</code> is a 114-line abstraction with exactly two functions, <code>save_bytes()</code> and <code>delete_by_url()</code> . Reimplement those with <code>boto3</code> and the rest of the codebase is untouched. Use an IRSA service account rather than static keys.
Supabase	Yes — keep it	This is the durable tier and it works from anywhere over HTTPS. The pods need outbound internet only.
Ephemeral	Yes — this is the answer	The container filesystem holds only the code, the committed demo assets and the rebuildable search index.

If you want to run it with no external dependencies at all

Leaving `DATABASE_URL` and `SUPABASE_*` unset makes the app fall back to local JSON files and local disk. That is a legitimate way to demo it in-cluster without a Supabase account — but then it *does* become stateful, and you

would need a ReadWriteOnce PVC and exactly one replica. Alternatively run Postgres in-cluster (any Postgres works, it is not Supabase-specific — see the note in section 7 about the pooler setting) and point `DATABASE_URL` at it.

5. Runtime characteristics that affect Kubernetes

The backend holds in-memory state that is not shared between replicas

The backend is stateless with respect to *durable data*, but three pieces of live state exist only inside a single process:

In-memory state	Where	Effect when running multiple replicas
WebSocket connection registry — <code>dict[email, list[WebSocket]]</code>	<code>routers/messages.py</code> lines 36–55	A message is pushed in real time only to sockets connected to <i>the same pod</i> . With two replicas, two users on different pods will not see each other's messages arrive live — the messages are still stored correctly and appear on refresh or poll.
SSE event bus — <code>list[asyncio.Queue]</code> of subscribers	<code>utils/events.py</code> lines 27–42	Admin live-monitoring events are only delivered to SSE clients attached to the pod that produced the event.
Semantic index, response cache, conversation memory	<code>utils/semantic_embeddings.py</code> , <code>utils/response_cache.py</code> , <code>utils/conversation_memory.py</code>	Harmless duplication — each pod builds its own. Costs memory and startup time, nothing more.

RECOMMENDATION

Start with **one replica**. It is the honest configuration for this codebase and everything works perfectly. If you want to scale out for the exercise, you have three options, in increasing order of effort: (1) accept degraded real-time chat across pods; (2) enable session affinity on the ingress so a user's HTTP and WebSocket traffic reliably land on one pod — this fixes the common case but not cross-user delivery between pods; (3) do it properly by replacing the in-process fan-out in `messages.py` and `events.py` with Redis pub/sub, which is a genuinely instructive change and touches only those two files.

Other runtime facts

- **Port:** the app reads `$PORT` at startup and falls back to `8000`. (*app.py line 203*)
- **Single process:** the start command runs one Uvicorn worker. Do not add `--workers N` without reading the note above — each worker is a separate process with its own connection registry and its own copy of the index, so it has the same split-brain effect as extra replicas, plus $N \times$ the memory.
- **Startup work:** the semantic index is built once during lifespan startup, before the app serves traffic. With an embeddings API configured this means one outbound API call per company/supplier at boot.
- **Background thread:** a daemon thread polls the source data every 10 seconds and incrementally refreshes the index. It is stopped cleanly on shutdown, so `SIGTERM` handling is fine; allow a few seconds of `terminationGracePeriodSeconds` (the default 30 is plenty).
- **No local ML models:** PyTorch, sentence-transformers, FAISS-building-from-scratch and EasyOCR are deliberately excluded from `requirements.txt` to fit in 512 MB. Do not add them back unless you raise the

memory limit above 2 GB.

6. Containerisation: reference Dockerfiles and build commands

To repeat the warning from page 1: **these files do not exist in the repository — you are writing them.** What follows mirrors exactly how Render and Vercel build and run the two parts today, so it is a faithful starting point rather than a guess.

Backend — Backend/Dockerfile

```
FROM python:3.11.9-slim

ENV PYTHONUNBUFFERED=1 \
    PYTHONDONTWRITEBYTECODE=1 \
    PIP_NO_CACHE_DIR=1

WORKDIR /app

# psycopg[binary] and PyMuPDF ship wheels, so no compiler toolchain is needed.
# curl is only here for the healthcheck; drop it if you probe from Kubernetes.
RUN apt-get update && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

# The app creates these at startup, but pre-creating them keeps a read-only
# root filesystem viable if you set one (mount emptyDir at these paths).
RUN mkdir -p uploads index_data company_data

EXPOSE 8000
ENV PORT=8000

CMD ["sh", "-c", "uvicorn app:app --host 0.0.0.0 --port ${PORT}"]
```

Build context is the `Backend/` directory, because the application resolves its data paths relative to its own root:

```
docker build -t scc-backend:1.0 ./Backend
docker run --rm -p 8000:8000 --env-file ./Backend/.env scc-backend:1.0
# verify: curl http://localhost:8000/api/health -> {"status":"ok","version":"1.0.0"}
```

Add a `Backend/.dockerignore` so secrets and junk never enter the image:

```
.env
.env.*
!.env.example
__pycache__/
*.py[cod]
.venv/
venv/
index_data/*
uploads/*
*.log
.git
```

Frontend — Frontend/Dockerfile

Two stages: build with Node, then serve the static output with nginx. Note that `VITE_API_URL` is compiled into the bundle at *build* time, not read at runtime — so it must be a build argument, and changing it requires rebuilding the image.

```
FROM node:20-alpine AS build
WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .

# Baked into the bundle at build time. Must be the browser-reachable backend
# origin (your ingress host), NOT an in-cluster service DNS name.
ARG VITE_API_URL=""
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:1.27-alpine
COPY --from=build /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

```
docker build -t scc-frontend:1.0 \
  --build-arg VITE_API_URL=https://api.your-domain.com ./Frontend
docker run --rm -p 8080:80 scc-frontend:1.0
```

Frontend — Frontend/nginx.conf

The SPA uses client-side routing, so every unknown path must fall back to `index.html`, exactly as the last rewrite rule in `vercel.json` does today:

```
server {
    listen 80;
    server_name _;
    root /usr/share/nginx/html;
    index index.html;

    # Hashed assets are immutable; index.html must never be cached.
    location /assets/ {
        add_header Cache-Control "public, max-age=31536000, immutable";
    }
    location = /index.html {
        add_header Cache-Control "no-store";
    }

    # SPA fallback – required for client-side routing.
    location / {
        try_files $uri $uri/ /index.html;
    }
}
```

Add a `Frontend/.dockerignore`:


```
node_modules
dist
.env
.env.*
!.env.example
.vercel
coverage
test-results
playwright-report
*.log
.git
```

Two routing models — pick one

Today, Vercel proxies `/api`, `/uploads` and `/company_data` to Render via `vercel.json` rewrites, which is why the browser never hits the backend origin for HTTP and no CORS configuration is needed. In Kubernetes you can reproduce either shape:

- **Same-origin (recommended, mirrors production).** One ingress host. Route `/api`, `/uploads` and `/company_data` to the backend service and everything else to the nginx frontend service. No CORS concerns, and the WebSocket travels over the same host. You can also do this inside nginx with `proxy_pass` instead of at the ingress.
- **Split origins.** Frontend and backend on separate hosts. Then you *must* set `CORS_ORIGINS` on the backend to the frontend's exact origin, and set `VITE_API_URL` at build time so the WebSocket can find the backend.

7. Environment variables reference

Everything is read from the environment; nothing is hardcoded. `.env` files are git-ignored and have never been committed — the repository contains no credentials of any kind, so your friend will need his own Supabase project and API keys, or he can run in local-JSON mode with none of them.

Backend

Variable	Secret	Required	Notes
ENV	No	Yes	Set to <code>production</code> . This enforces a strong <code>JWT_SECRET_KEY</code> and disables uvicorn autoreload.
PORT	No	No	Defaults to 8000. Set it if your container port differs.
JWT_SECRET_KEY	Yes	Yes	Long random string. Generate: <code>python -c "import secrets; print(secrets.token_hex(32))"</code> . Must be identical across replicas or tokens break on every pod hop.
CORS_ORIGINS	No	Conditional	Comma-separated exact origins. Only needed if the browser calls the API on a different origin than the page.
DATABASE_URL	Yes	For durability	Supabase transaction-pooler URI (<code>...pooler.supabase.com:6543</code>). Empty $\Rightarrow$ local JSON files.
SUPABASE_URL	No	For durability	<code>https://<project-ref>.supabase.co</code>
SUPABASE_SERVICE_ROLE_KEY	Yes	For durability	Must be the <i>legacy</i> <code>service_role</code> JWT starting <code>eyJ...</code> . The newer opaque <code>sb_secret_...</code> format is rejected by Supabase Storage's REST API with "Invalid Compact JWS". Backend-only — never expose it to a browser.
SUPABASE_BUCKET	No	No	Defaults to <code>media</code> . The bucket must exist and be public.
OPENROUTER_API_KEY1...N	Yes	For AI chat	Every variable whose name starts with <code>OPENROUTER_API_KEY</code> is auto-discovered and rotated on rate limits.
EMBEDDINGS_API_URL / EMBEDDINGS_API_KEY / EMBEDDINGS_MODEL	Key: yes	No	Currently Cohere. Omit entirely and search silently degrades to BM25 keyword matching — the app still works and costs nothing.
DB_POOL_MAX	No	No	Connection-pool ceiling, default 5. Relevant if you scale replicas — total connections are replicas $\times$ this value.
LOG_LEVEL	No	No	<code>INFO</code> by default.

Put the four secret values in a Kubernetes `Secret` (or External Secrets / AWS Secrets Manager) and the rest in a `ConfigMap`. Fully documented defaults live in `Backend/.env.example`.

Frontend

Variable	Secret	When	Notes
VITE_API_URL	No — public	Build time	The browser-reachable backend origin. Required for real-time chat when the frontend and backend are on different origins, because HTTP is proxied but WebSockets are not. Leave unset for same-origin ingress routing.

IMPORTANT

Anything prefixed `VITE_` is compiled into the JavaScript bundle and is therefore **publicly visible to anyone who opens the site**. Never put a secret in a `VITE_` variable, and never inject one into the frontend container at runtime expecting it to be private.

A note on non-Supabase Postgres

Nothing in the data layer is Supabase-specific — it is plain Postgres with a single JSONB table that the app creates itself on boot. You can point `DATABASE_URL` at RDS or an in-cluster Postgres and it will work. One detail: the connection sets `prepare_threshold = None` to disable server-side prepared statements for pgBouncer transaction-mode compatibility. That is harmless on a direct connection, just very slightly slower.

8. Networking, routing and ingress

Routes exposed by the backend

Path	Type	Purpose
<code>/api/auth/*</code>	HTTP	Registration, login, JWT issuance
<code>/api/companies/*</code> , <code>/api/suppliers/*</code> , <code>/api/requests/*</code>	HTTP	Marketplace and project requests
<code>/api/admin/*</code>	HTTP	Admin console (guarded by an admin-role dependency)
<code>/api/ai/*</code>	HTTP	AI assistant — the slowest endpoints; allow a generous proxy read timeout
<code>/api/upload/*</code>	HTTP	File uploads — set the ingress body-size limit to at least 10 MB (see below)
<code>/api/messages/*</code>	HTTP	Conversations and message history
<code>/api/messages/ws/{email}</code>	WebSocket	Real-time chat. Authenticated by a <code>?token=</code> query parameter.
<code>/api/events*</code>	SSE	Admin live monitoring — a long-lived streaming response
<code>/api/health</code>	HTTP	Health check — returns <code>{"status":"ok","version":"1.0.0"}</code>
<code>/docs</code>	HTTP	Swagger UI (FastAPI). Consider blocking this at the ingress in a public deployment.
<code>/company_data/*</code> , <code>/uploads/*</code>	Static	Legacy/demo assets served from the image

Ingress settings that actually matter

- **WebSocket upgrade.** Must be allowed on `/api/messages/ws/`. AWS ALB supports this natively; with the nginx ingress controller it works by default, but raise `proxy-read-timeout` (e.g. 3600) or idle chat sockets will be cut.

- **SSE buffering.** The `/api/events` stream must not be buffered, or admin events arrive in bursts or not at all. With nginx set `proxy_buffering off` for that path.
- **Upload body size.** Defaults are 5 MB for images and 10 MB for documents (`MAX_IMAGE_SIZE` , `MAX_DOC_SIZE`). The ingress limit must be at least as large — nginx's 1 MB default will reject uploads with a 413.
- **AI endpoint latency.** Calls to OpenRouter can take tens of seconds. A 30-second proxy timeout will produce spurious 504s.
- **Session affinity.** Only relevant if you run more than one replica — see section 5.

Egress

The backend pod needs outbound HTTPS to: Supabase (Postgres on 6543 and Storage on 443), `openrouter.ai` , and `api.cohere.ai` . In a private-subnet EKS setup that means a NAT gateway. If you apply NetworkPolicies, allow those destinations or the app will boot but every AI and persistence call will fail.

9. Health checks, probes and resources

Probe	Endpoint	Suggested settings
Liveness	GET <code>/api/health</code>	<code>periodSeconds: 20</code> , <code>failureThreshold: 3</code>
Readiness	GET <code>/api/health</code>	<code>periodSeconds: 10</code> , <code>failureThreshold: 3</code>
Startup	GET <code>/api/health</code>	<code>failureThreshold: 30</code> , <code>periodSeconds: 5</code> — gives up to 150 s for index building

WHY A STARTUP PROBE MATTERS HERE

The semantic index is built during FastAPI's lifespan startup, *before* the server accepts requests. With an embeddings API configured, that means one outbound call per company and supplier at boot. Without a startup probe, an aggressive liveness probe can kill the pod mid-build and put it in a restart loop that never completes. A startup probe with a generous `failureThreshold` avoids this cleanly.

Resource sizing

Workload	Requests	Limits	Notes
Backend	200m CPU / 512Mi	1000m CPU / 1Gi	Runs today within Render's 512 MB free tier, so 1Gi is comfortable headroom. Raise well above 2Gi only if you re-enable local PyTorch embeddings and OCR.
Frontend (nginx)	50m CPU / 64Mi	200m CPU / 128Mi	Serving static files only — very light.

The frontend is genuinely stateless and horizontally scalable without caveats; put an HPA on it freely. Apply the section 5 caveats before scaling the backend.

10. Known gotchas

1. **Silent fallback when the database connection fails.** If `DATABASE_URL` is set but unreachable, the app logs an error and *continues* using local JSON files instead of crashing. On an ephemeral pod this looks like it is working while quietly losing data on every restart. Check the startup logs for `"Database enabled – using Postgres document store"` before believing persistence is active. (*db.py lines 84–92*)
 2. **Same silent fallback for storage.** If the Supabase variables are missing, uploads go to the pod's local disk and vanish on restart, with no error surfaced to the user.
 3. **The Supabase service-role key must be the legacy JWT.** The newer `sb_secret_...` format fails with "Invalid Compact JWS" against the Storage REST API. Take it from the "Legacy anon, service_role API keys" tab.
 4. **Use the transaction pooler, not the direct connection.** Supabase's direct host (`db.<ref>.supabase.co:5432`) is often IPv6-only on free projects. Use `...pooler.supabase.com:6543`.
 5. **Free Supabase projects pause after about 7 days idle.** The database becomes unreachable and the app falls back to local files per gotcha 1 — a confusing failure mode a week after setup.
 6. **VITE_API_URL is baked in at build time.** It cannot be changed by editing a ConfigMap; changing the backend URL means rebuilding the frontend image.
 7. **The service name in URLs is spelled "contruction"** (missing the "s") in the live Render and Vercel hostnames. That is the real, working spelling — not a typo in this document.
 8. **/docs is public** by default. Block it at the ingress if this becomes internet-facing.
 9. **The bundled dataset is fictional.** Everything under `Backend/Database/`, `Backend/company_data/` and `Frontend/src/data/` is seeded demo content — companies, suppliers, contact details, chat history. There is no real personal data and no credential in the repository.
 10. **Do not add `--workers N` to the start command** as a quick scaling win. Each worker duplicates the in-memory WebSocket registry and the search index, giving the same split-brain behaviour as extra replicas plus $N \times$ the memory.
-

Quick reference

Question	Answer
Are there Dockerfiles?	No — none anywhere in the repository. Section 6 provides them.
Is <code>Database/</code> the production DB?	No. Supabase Postgres is, via a single <code>documents</code> table.
Does <code>company_data/</code> serve from the container?	Committed demo assets yes (read-only, from the image). New uploads no — Supabase Storage.
Does <code>uploads/</code> receive runtime writes?	Not in production. Supabase Storage does.
Is <code>index_data/</code> a rebuildable cache?	Yes, entirely. Per-pod, ephemeral, never shared.
EBS, S3, Supabase or ephemeral?	Ephemeral filesystem, with Supabase as the durable tier. No PVC required.
Deployment or StatefulSet?	Deployment . Start with one backend replica.
Backend container port	8000 (reads <code>\$PORT</code>).
Health endpoint	<code>GET /api/health</code>
Python / Node versions	Python 3.11.9 (pinned in <code>runtime.txt</code>); Node 20 for the frontend build.

All statements in this document were verified against the source code on branch `main`. Primary references: `Backend/app.py`, `Backend/backend/utils/data_handler.py`, `Backend/backend/utils/db.py`, `Backend/backend/utils/storage.py`, `Backend/backend/utils/index_sync.py`, `Backend/backend/routers/messages.py`, `Backend/backend/utils/events.py`, `Backend/render.yaml`, `Frontend/vercel.json`, `Frontend/vite.config.ts`.